

## Experiment 5

### Comprehensive Study of CNN Training, Regularization, Optimization, Hyperparameter Tuning, Transfer Learning and Cross-Validation

|                     |                                               |                |
|---------------------|-----------------------------------------------|----------------|
| Degree & Branch     | B.Tech Artificial Intelligence & Data Science | Semester<br>V  |
| Subject Code & Name | CS3807 – Deep Learning Laboratory             | AY:<br>2026–27 |

## 1. Objective

The objective of this experiment is to systematically study the effect of weight initialization, regularization, optimization algorithms, CNN hyperparameters, transfer learning, fine-tuning and cross-validation on image classification performance.

The experiment uses a single CNN architecture, MobileNetV2, and the Oxford-IIIT Pet dataset. Students will analyze the effect of individual design choices using training/validation curves and finally select a suitable configuration using 5-fold cross-validation.

## 2. Learning Outcomes

After completing this experiment, students will be able to:

- explain different weight initialization strategies;
- identify and analyze overfitting using training and validation curves;
- compare SGD, Momentum, RMSProp and Adam;
- explain CNN hyperparameters such as kernel size, stride, padding and pooling;
- understand Batch Normalization and Dropout;
- perform transfer learning and fine-tuning using MobileNetV2;
- tune important training hyperparameters;
- apply 5-fold cross-validation for model selection; and
- justify the final model using accuracy, variability and computational cost.

## 3. Dataset and Experimental Setup

### Dataset: Oxford-IIIT Pet Dataset

The Oxford-IIIT Pet Dataset contains images of cats and dogs belonging to 37 breeds. The images are RGB and have different spatial dimensions. For this experiment, all images are resized to

$$224 \times 224 \times 3.$$

The dataset is suitable for demonstrating transfer learning using ImageNet-pretrained CNNs.

### Experimental considerations:

- Use RGB images resized to  $224 \times 224$ .
- Normalize the input images according to the pretrained model requirements.
- Maintain separate training, validation and test data.
- The test set must remain untouched during hyperparameter selection.
- Use MobileNetV2 because it is computationally lighter than many large CNN architectures and is more suitable for CPU-based laboratory work.

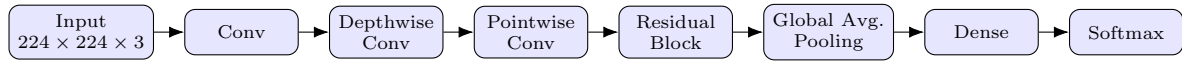
## 4. MobileNetV2 Architecture

MobileNetV2 is a lightweight CNN architecture designed to reduce computational complexity while maintaining good image representation.

Its important components include:

- depthwise convolution;
- pointwise  $1 \times 1$  convolution;
- inverted residual blocks;
- linear bottlenecks;
- batch normalization; and
- ReLU6 activation.

A simplified representation is:



**Note:** The above diagram is a simplified conceptual representation rather than the complete MobileNetV2 architecture.

## 5. Weight Initialization

Weight initialization determines the initial values of trainable weights before training begins. A suitable initialization can improve convergence and reduce problems such as vanishing or exploding activations.

Students should investigate:

- Zero initialization
- Random initialization
- Xavier/Glorot initialization
- He initialization

### Expected Analysis

Students should compare the initialization strategies using:

#### Plot 1: Training Loss vs. Epoch

- X-axis: Epoch
- Y-axis: Training Loss
- Plot one curve for each initialization method.

#### Plot 2: Validation Accuracy vs. Epoch

- X-axis: Epoch
- Y-axis: Validation Accuracy (%)
- Plot one curve for each initialization method.

Students should identify which initialization gives faster and more stable convergence.

## 6. Regularization and Overfitting

Overfitting occurs when a model performs very well on the training data but generalizes poorly to unseen data.

Students should compare:

- No regularization
- L2 regularization
- Dropout
- Batch Normalization

### Expected Plots

#### Plot 3: Training and Validation Accuracy vs. Epoch

- X-axis: Epoch
- Y-axis: Accuracy (%)
- Plot separate curves for training and validation accuracy.

Students should identify the generalization gap.

#### Plot 4: Training and Validation Loss vs. Epoch

- X-axis: Epoch
- Y-axis: Loss
- Plot training and validation loss.

Students should identify whether validation loss begins to increase while training loss continues to decrease.

## 7. Batch Normalization

Batch Normalization normalizes activations within a mini-batch during training.

For a mini-batch

$$x_1, x_2, \dots, x_m,$$

the batch mean is

$$\mu_B = \frac{1}{m} \sum_{i=1}^m x_i$$

and the batch variance is

$$\sigma_B^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu_B)^2.$$

The normalized activation is

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}.$$

The final output is

$$y_i = \gamma \hat{x}_i + \beta,$$

where  $\gamma$  and  $\beta$  are learnable parameters.

### Numerical Example

Consider

$$x = [2, 4, 6, 8].$$

The mean is

$$\mu_B = \frac{2 + 4 + 6 + 8}{4} = 5.$$

The variance is

$$\sigma_B^2 = \frac{(2-5)^2 + (4-5)^2 + (6-5)^2 + (8-5)^2}{4} = 5.$$

Therefore,

$$\sqrt{5} \approx 2.236.$$

Ignoring the very small  $\epsilon$ ,

$$\hat{x}_1 = \frac{2-5}{2.236} \approx -1.342,$$

$$\hat{x}_2 = \frac{4-5}{2.236} \approx -0.447,$$

$$\hat{x}_3 = \frac{6-5}{2.236} \approx 0.447,$$

$$\hat{x}_4 = \frac{8-5}{2.236} \approx 1.342.$$

Hence,

$$\hat{x} \approx [-1.342, -0.447, 0.447, 1.342]$$

If  $\gamma = 1$  and  $\beta = 0$ , these are the final outputs.

**Inference:** Batch Normalization produces approximately zero-centred and unit-scaled activations for this mini-batch. The learnable  $\gamma$  and  $\beta$  allow the network to learn an appropriate scale and shift.

**Plot 5: With vs. Without Batch Normalization**

- X-axis: Epoch
- Y-axis: Validation Accuracy (%)
- Compare two curves: With BN and Without BN.

## 8. Optimization Algorithms

Students should compare the following optimization methods:

- SGD
- Momentum
- RMSProp
- Adam

The objective is to study convergence speed, stability and final validation performance.

**Plot 6: Training Loss vs. Epoch for Different Optimizers**

- X-axis: Epoch
- Y-axis: Training Loss
- One curve for each optimizer.

**Plot 7: Validation Accuracy vs. Epoch for Different Optimizers**

- X-axis: Epoch
- Y-axis: Validation Accuracy (%)
- One curve for each optimizer.

| Optimizer | Final Loss | Best Val. Accuracy | Epoch to Converge | Time |
|-----------|------------|--------------------|-------------------|------|
| SGD       |            |                    |                   |      |
| Momentum  |            |                    |                   |      |
| RMSProp   |            |                    |                   |      |
| Adam      |            |                    |                   |      |

## 9. CNN Hyperparameter Tuning

The following hyperparameters should be investigated:

| Hyperparameter            | Suggested Values                 |
|---------------------------|----------------------------------|
| Learning Rate             | 0.001, 0.0001                    |
| Batch Size                | 16, 32, 64                       |
| Dropout Rate              | 0, 0.25, 0.5                     |
| Optimizer                 | SGD, Adam                        |
| Fine-Tuning Learning Rate | $10^{-4}$ , $10^{-5}$            |
| Frozen Layers             | Frozen base / Partial unfreezing |

For CNN concepts, students should also understand:

- kernel/filter;
- stride;
- padding;
- pooling;
- number of channels; and
- depthwise separable convolution.

For a convolution operation, the output dimension can be calculated as

$$O = \left\lfloor \frac{N + 2P - K}{S} \right\rfloor + 1,$$

where  $N$  is input size,  $K$  is kernel size,  $P$  is padding and  $S$  is stride.

**Experimental rule:** During a controlled hyperparameter study, change one hyperparameter at a time while keeping the remaining settings fixed.

### Plot 8: Learning Rate vs. Validation Accuracy

- X-axis: Learning Rate
- Y-axis: Validation Accuracy (%)

### Plot 9: Batch Size vs. Validation Accuracy

- X-axis: Batch Size
- Y-axis: Validation Accuracy (%)

### Plot 10: Dropout Rate vs. Validation Accuracy

- X-axis: Dropout Rate
- Y-axis: Validation Accuracy (%)

## 10. Transfer Learning and Fine-Tuning

MobileNetV2 pretrained on ImageNet should be used.

### Case A: Feature Extraction

Pretrained MobileNetV2 → Freeze Base → New Classifier

The pretrained convolutional layers are frozen and only the newly added classification layers are trained.

### Case B: Fine-Tuning

Pretrained MobileNetV2 → Unfreeze Selected Layers → Small Learning Rate

The upper layers of the pretrained network are unfrozen and trained with a smaller learning rate.

### Plot 11: Feature Extraction vs. Fine-Tuning

- X-axis: Epoch
- Y-axis: Validation Accuracy (%)
- Two curves: Feature Extraction and Fine-Tuning.

### Plot 12: Training and Validation Loss

Students should compare the loss curves before and after fine-tuning.

**Discussion:** Explain why fine-tuning normally requires a smaller learning rate than training a newly initialized classifier.

## 11. K-Fold Cross-Validation

After the individual studies, select 3–4 promising configurations.

Use 5-fold cross-validation on the training data.

For  $K = 5$ ,

$$\bar{A} = \frac{1}{5} \sum_{i=1}^5 A_i$$

and

$$SD = \sqrt{\frac{1}{5} \sum_{i=1}^5 (A_i - \bar{A})^2}.$$

Report the result as

$$\boxed{\bar{A} \pm SD}.$$

The independent test set must not be used during hyperparameter selection.

| Configuration | F1 | F2 | F3 | F4 | F5 | Mean $\pm$ SD |
|---------------|----|----|----|----|----|---------------|
| C1            |    |    |    |    |    |               |
| C2            |    |    |    |    |    |               |
| C3            |    |    |    |    |    |               |
| C4            |    |    |    |    |    |               |

### Plot 13: 5-Fold Cross-Validation Accuracy

- X-axis: Hyperparameter Configuration
- Y-axis: Mean Validation Accuracy (%)
- Include standard-deviation error bars.

Students should discuss both mean performance and variability.

## 12. Final Model Evaluation

After selecting the best configuration using cross-validation:

1. Retrain the selected configuration using the complete training data.
2. Keep the independent test set untouched until this stage.
3. Evaluate the final model on the test set.

Report:

| Metric                | Value |
|-----------------------|-------|
| Mean CV Accuracy      |       |
| CV Standard Deviation |       |
| Test Accuracy         |       |
| Precision             |       |
| Recall                |       |
| F1-score              |       |
| Training Time         |       |
| Number of Parameters  |       |

### Plot 14: Confusion Matrix

Students should identify:

- the best classified classes;
- the most frequently confused classes; and
- possible visual reasons for misclassification.

### Optional Plot 15: Misclassified Images

Display representative incorrectly classified images and provide a brief explanation for each.

## 13. Source Code

The complete source code is attached separately in the GitHub repository and submitted along with this report.

### GitHub Repository:

<https://github.com/Sadhana2006-art/Deep-Learning-Lab>

## 14. Results

Optimization Algorithms:

| Optimizer | Final Loss | Best Val. Accuracy | Epoch to Converge | Time   |
|-----------|------------|--------------------|-------------------|--------|
| SGD       | 0.1402     | 91.85%             | 10                | 183.04 |
| Momentum  | 0.0074     | 92.39%             | 9                 | 253.35 |
| RMSProp   | 0.0016     | 92.26%             | 9                 | 258.68 |
| Adam      | 0.0053     | 92.80%             | 10                | 267.21 |

K-Fold Cross-Validation:

| Configuration | F1     | F2     | F3     | F4     | F5     | Mean $\pm$ SD      |
|---------------|--------|--------|--------|--------|--------|--------------------|
| C1            | 91.34% | 90.66% | 91.68% | 92.36% | 92.52% | 91.71% $\pm$ 0.68% |
| C2            | 91.00% | 92.19% | 91.51% | 91.85% | 92.18% | 91.75% $\pm$ 0.45% |
| C3            | 91.00% | 91.34% | 91.68% | 91.85% | 92.52% | 91.68% $\pm$ 0.51% |
| C4            | 83.87% | 84.89% | 86.42% | 87.10% | 88.95% | 86.24% $\pm$ 1.76% |

#### Final Model Evaluation:

| Metric                | Value     |
|-----------------------|-----------|
| Mean CV Accuracy      | 91.75%    |
| CV Standard Deviation | 0.45%     |
| Test Accuracy         | 89.42%    |
| Precision             | 89.79%    |
| Recall                | 89.42%    |
| F1-score              | 89.37%    |
| Training Time         | 162.81 s  |
| Number of Parameters  | 2,426,725 |

#### Overall Results:

| Configuration                             | CV Accuracy | SD    | Test Accuracy | Training Time |
|-------------------------------------------|-------------|-------|---------------|---------------|
| Baseline                                  | 92.7%       | –     | 92.7%         | 201.47 s      |
| Best Initialization (Xavier/Glorot)       | 93.1%       | –     | 93.1%         | 254.81 s      |
| Best Regularization (Batch Normalization) | 92.3%       | –     | 92.3%         | 211.91 s      |
| Best Optimizer (Adam)                     | 92.7%       | –     | 92.7%         | 267.21 s      |
| Best Hyperparameters (C2)                 | 91.75%      | 0.45% | 89.42%        | 162.81 s      |
| Fine-Tuned Model                          | 90.22%      | –     | 90.22%        | 259.18 s      |

The configuration C2 was chosen because it had the best average 5 fold cross-validation accuracy at 91.75%, and it also had the least standard deviation at 0.45%. This means that it was not only an accurate configuration, but also it produced similar results across all folds, thus consistency of results. C2 took the least time to train at 162.81 seconds among the other main configurations. After re-training on the full training set, C2 gave an independent test accuracy of 89.42% with Precision, Recall and F1-score being 89.79%, 89.42% and 89.37%, respectively.

## 15. Plots with Inference

### 15.1 Training Loss vs Epoch for Different Weight Initializations

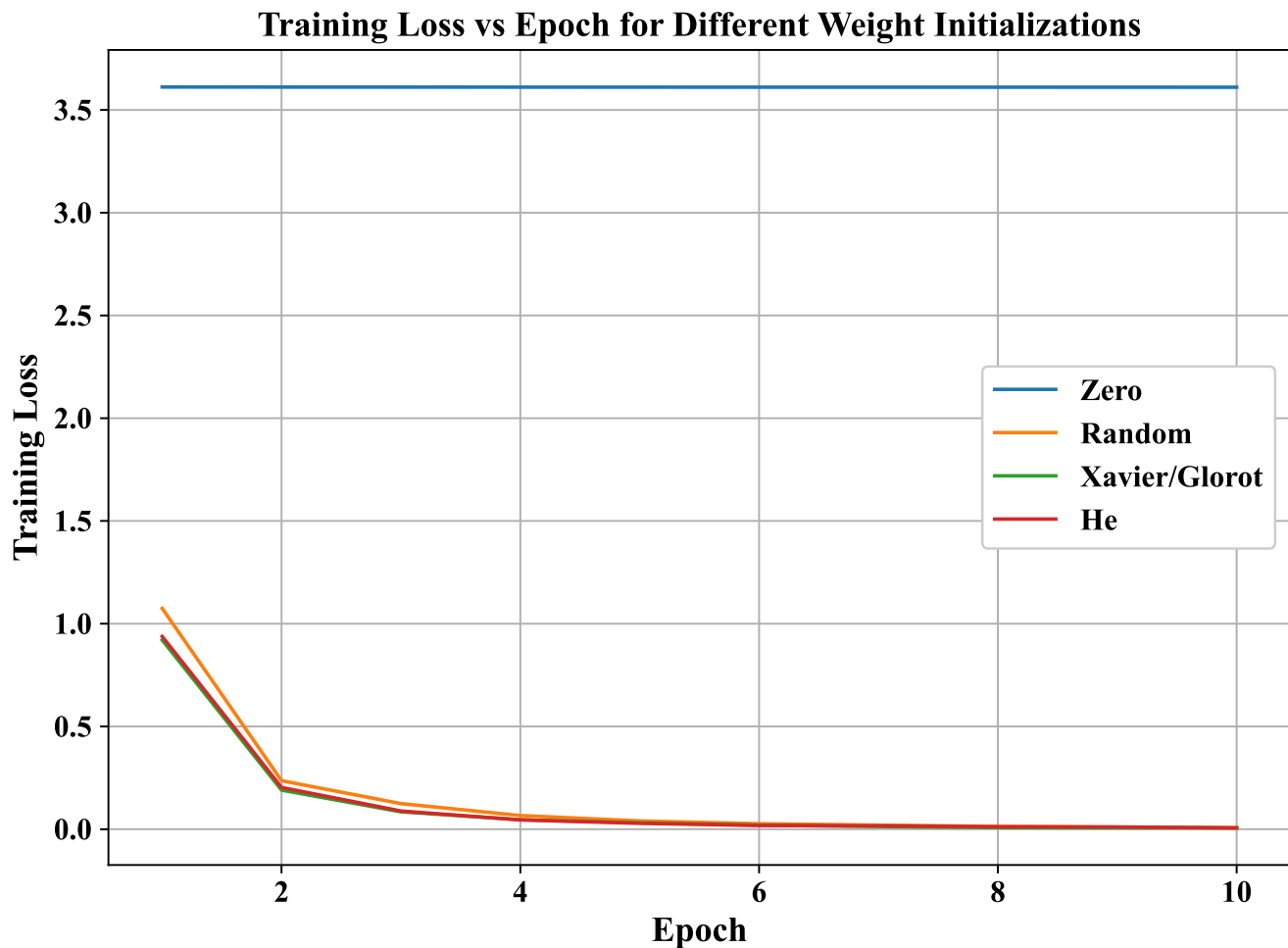


Figure 1: Training Loss vs Epoch

#### Interpretation:

The initialization curve for the zero initializer remains relatively constant at 3.61, while that of random, Xavier/Glorot, and He goes down to zero very quickly. The zero initializer fails to provide a good learning process because all neurons have equal weights, hence keeping the training loss constant. The random, Xavier/Glorot, and He initializers break symmetry and enable quick minimization of the training loss.



## 15.2 Validation Accuracy vs Epoch for Different Weight Initializations

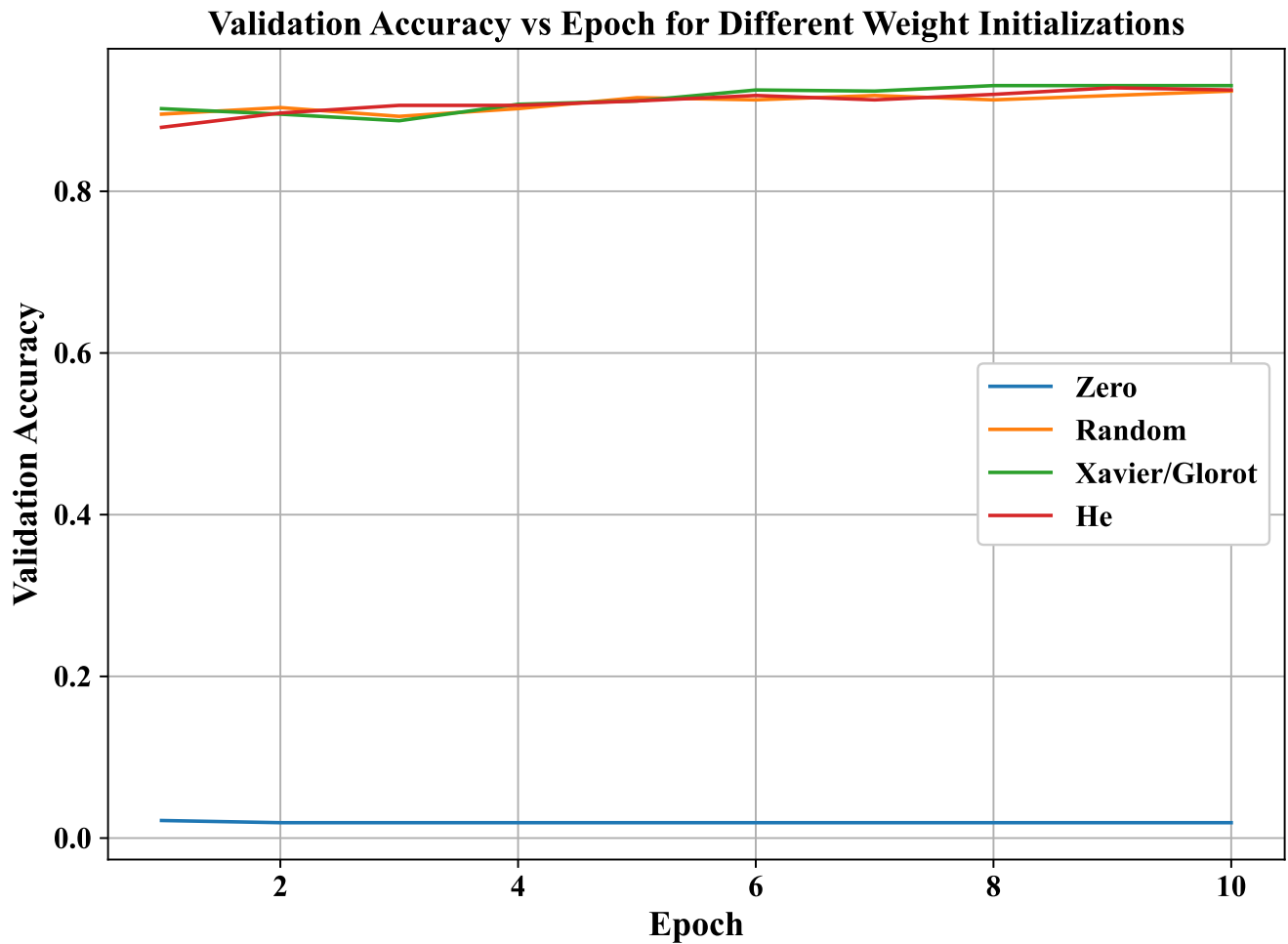


Figure 2: Validation Accuracy vs Epoch

### Interpretation:

Zero initialization stays at about 1.9%, which is close to the chance rate of 37 classes. The other three initialization methods quickly get around 90-93% validation accuracy. Zero initialization is unable to learn meaningful class representations and therefore gets nearly the chance-level validation accuracy. Random, Xavier/Glorot, and He initialization successfully break the symmetry, and Xavier/Glorot reaches the best final validation accuracy of approximately 93.1%.

### 15.3 Training and Validation Accuracy vs. Epoch

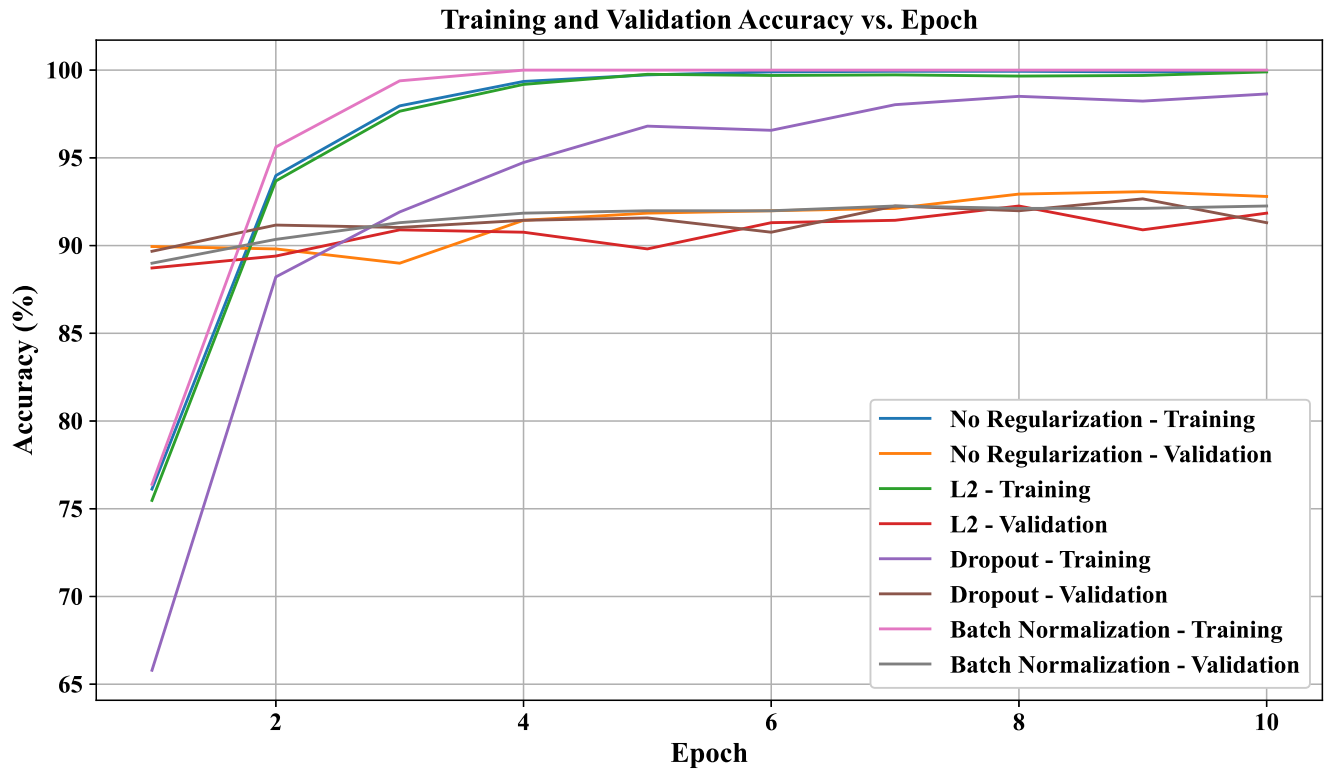


Figure 3: Training and Validation Accuracy vs Epoch

#### Interpretation:

The critical thing to note is the generalization gap:

Without Regularization: The training accuracy is nearly 100%, but the validation remains at 93%. With L2: Training accuracy is around 99.9%, and validation accuracy is about 92.5%. With Dropout: Training accuracy is nearly 98.6%, but validation accuracy is 91.3%. With Batch Normalization: Training accuracy is 100%, and validation accuracy is 92.3%. This shows a clear gap between training and validation accuracy, indicating some degree of overfitting.

## 15.4 Training and Validation Loss vs. Epoch

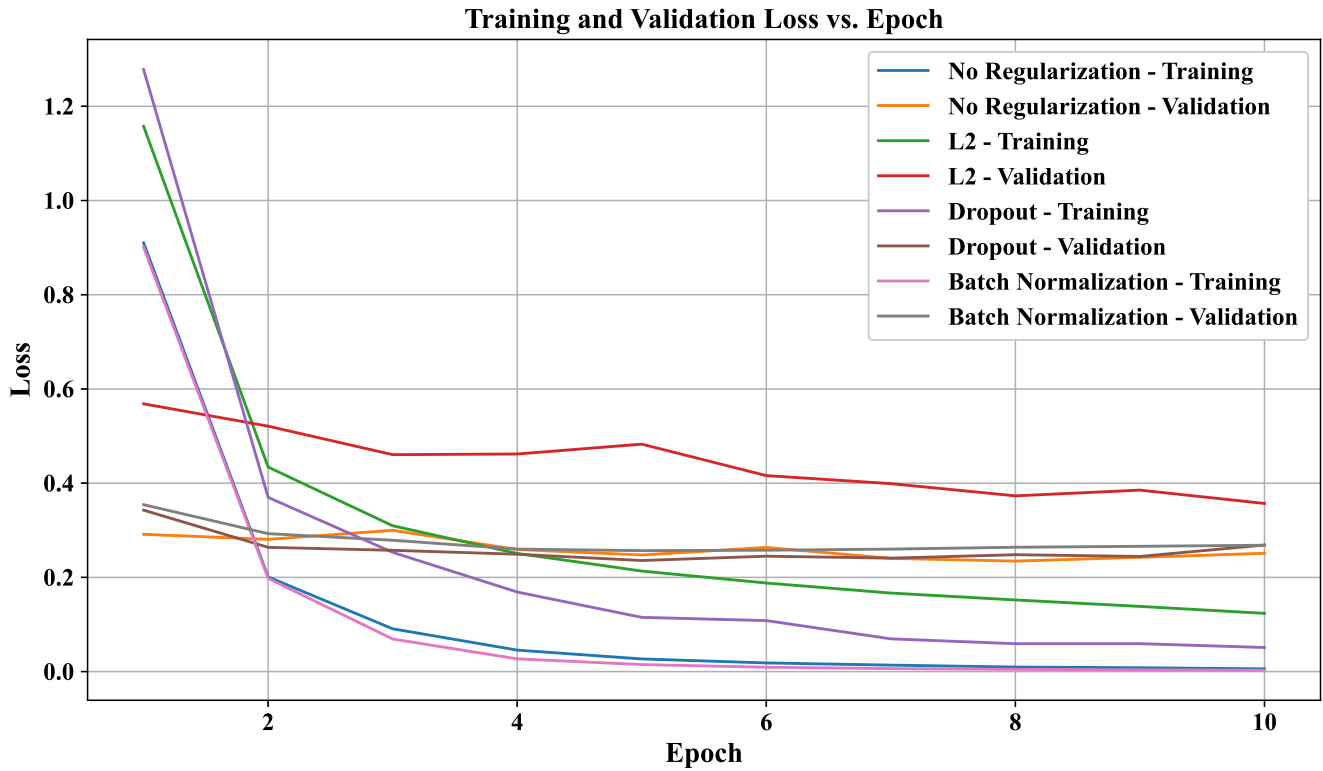


Figure 4: Training and Validation Loss vs Epoch

### Interpretation:

Yes, that is what happens in Batch Normalization: Loss during training keeps reducing from approximately 0.90  $\rightarrow$  0.003 Loss during validation reduces up to approximately 0.236 at epoch 5 After this, loss during validation begins to increase/fluctuate: Epoch 6: 0.258 Epoch 7: 0.260 Epoch 8: 0.264 Epoch 9: 0.266 This is an indication of overfitting.

## 15.5 Validation Accuracy With vs. Without Batch Normalization

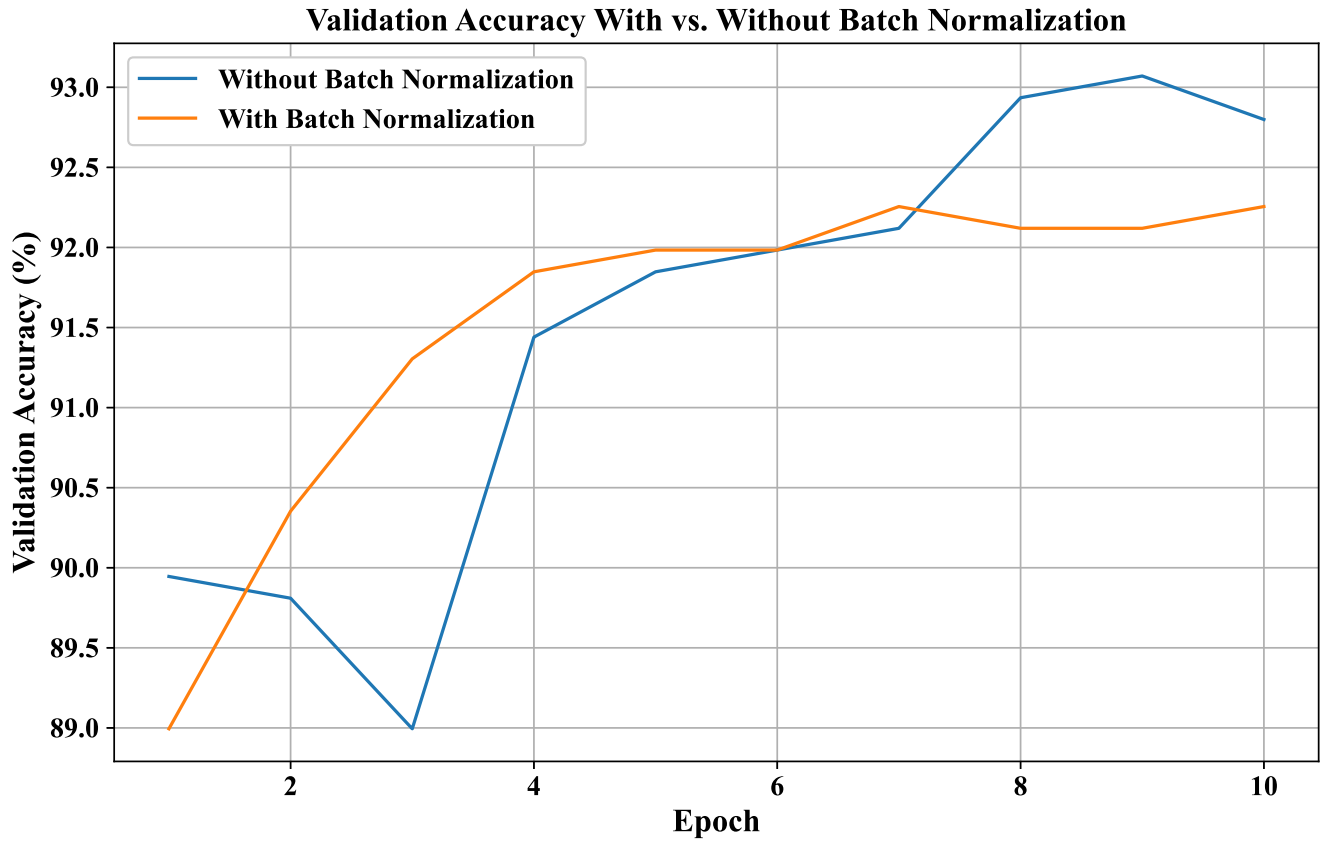


Figure 5: Validation Accuracy With vs. Without Batch Normalization

### Interpretation:

However, batch normalization causes faster increases in validation accuracy in the early epochs. The model that uses batch normalization achieves 92% accuracy in validation faster than the model without batch normalization. Nevertheless, in about epoch 7, the model without batch normalization overtakes the model with batch normalization by achieving the maximum validation accuracy of about 93.1%. Therefore, batch normalization helps to learn fast and consistently, but it does not guarantee higher validation accuracy at the end of learning.

## 15.6 Training Loss vs. Epoch for Different Optimizers

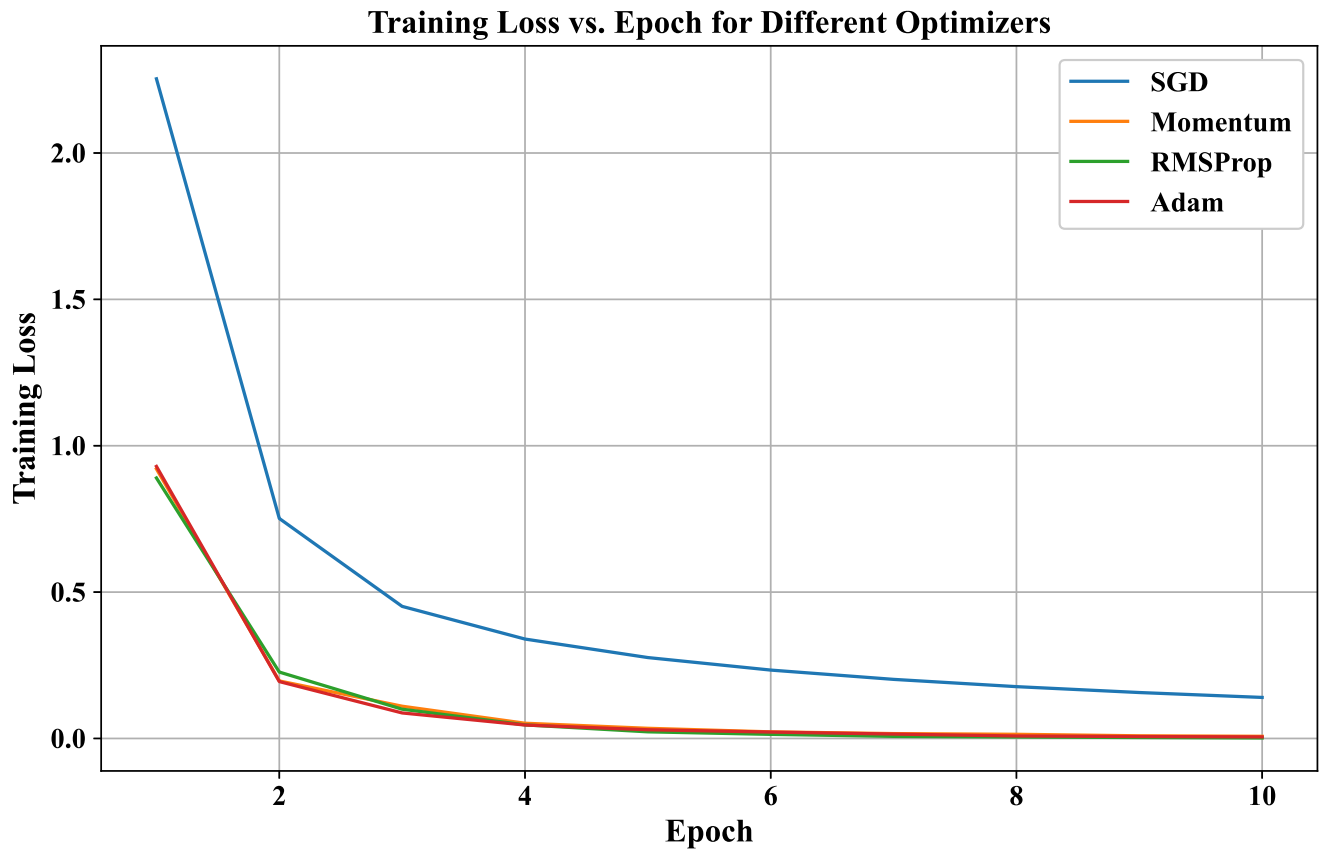


Figure 6: Training Loss vs Epoch for Different Optimizers

### Interpretation:

Training loss of the optimizers like SGD, Momentum, RMSProp, and Adam is plotted in the graph. Training loss reduces for all optimizers; however, in the case of RMSProp and Adam, the loss decreases much rapidly compared to the SGD. The adaptive gradient descent is much better than simple SGD.

## 15.7 Validation Accuracy vs. Epoch for Different Optimizers

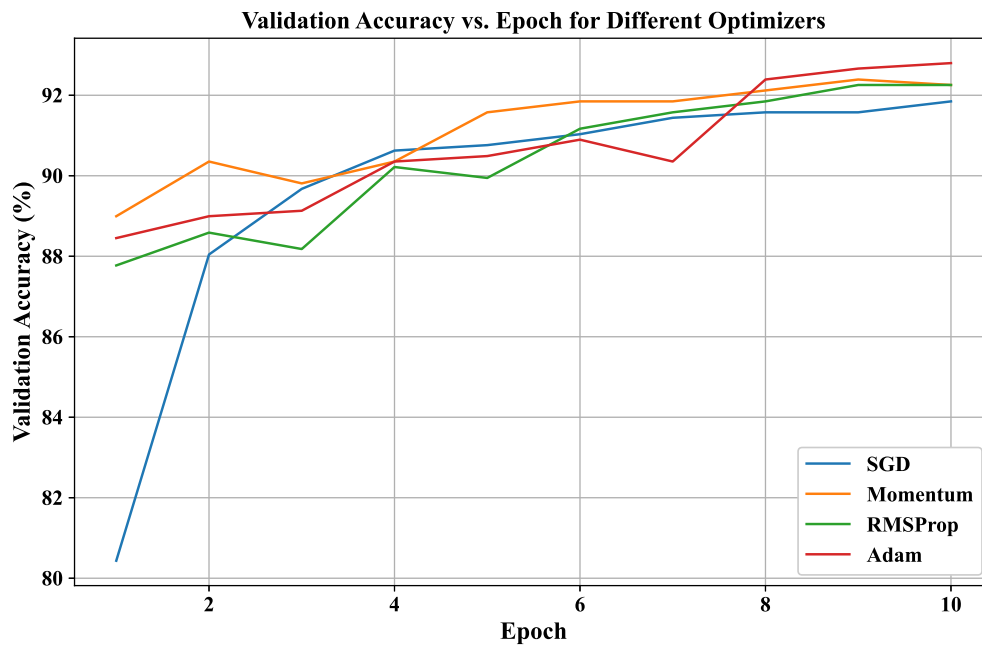


Figure 7: Validation Accuracy vs Epoch for Different Optimizers

### Interpretation:

The figure depicts the comparison of validation accuracy among the four optimization methods. Adam exhibits the best performance with 92.8% validation accuracy, followed by Momentum and RMSProp, whereas SGD gets up to 91.8%. This difference arises due to the use of different techniques for updating the parameters.

## 15.8 Learning Rate vs. Validation Accuracy

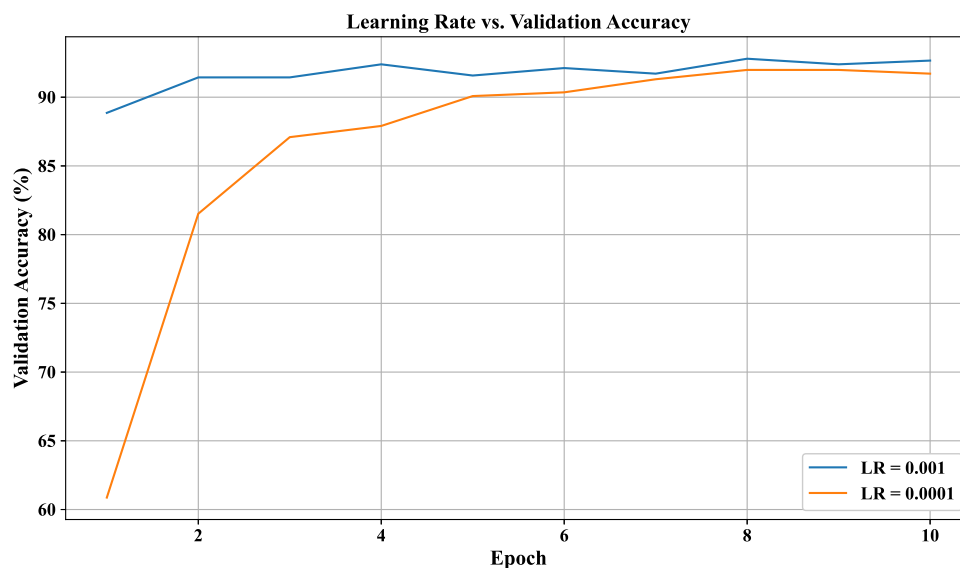


Figure 8: Learning Rate vs. Validation Accuracy

### Interpretation:

The plot shows the comparison of validation accuracies of learning rates 0.001 and 0.0001. The learning rate of 0.001 gives better validation accuracy compared to that of 0.0001 (validation accuracy of 92.8% versus 91.7%, respectively). The reason for this could be that 0.001 gives a more efficient update to parameters in limited training epochs.

## 15.9 Batch Size vs. Validation Accuracy

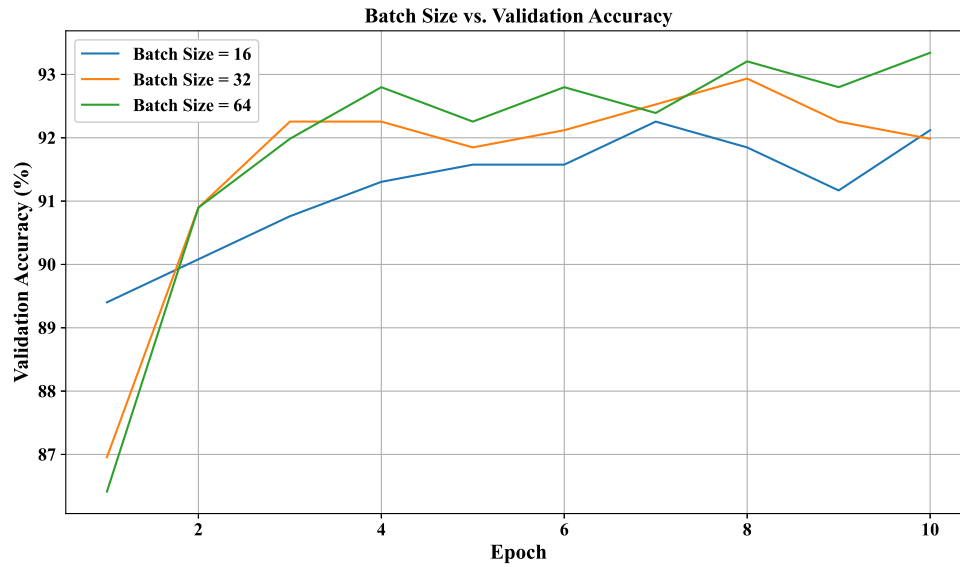


Figure 9: Batch Size vs. Validation Accuracy

### Interpretation:

The graph illustrates the accuracy of validation using different batch sizes such as 16, 32, and 64. The batch size of 64 has attained the highest validation accuracy at around 93.3% compared to the other sizes. The bigger batch size makes the gradient estimation process more stable.

## 15.10 Dropout Rate vs. Validation Accuracy

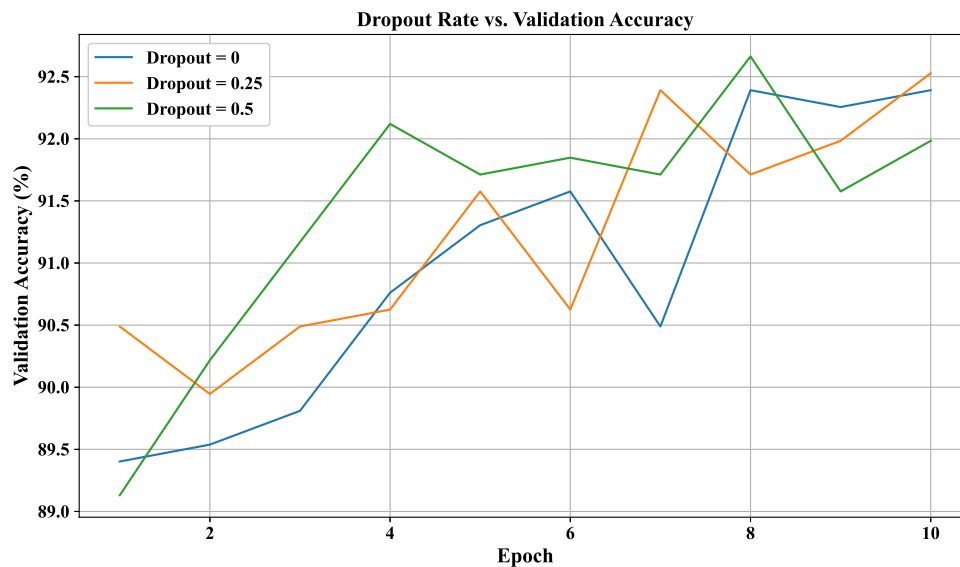


Figure 10: Dropout Rate vs. Validation Accuracy

### Interpretation:

The plot illustrates the validation accuracy in relation to the following dropout values: 0, 0.25 and 0.5. For a dropout value of 0.5, there is the highest validation accuracy achieved ( 92.7%). The increased regularization may help in avoiding overfitting through limiting the reliance on specific neurons.

## 15.11 Feature Extraction vs. Fine-Tuning

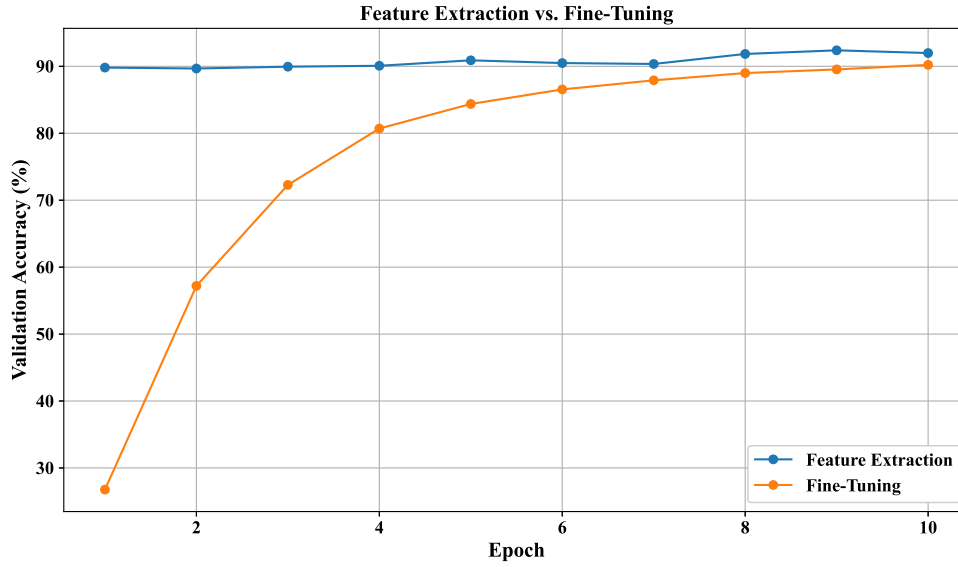


Figure 11: Feature Extraction vs. Fine-Tuning

### Interpretation:

Validation accuracy performance of feature extraction vs fine-tuning is represented on the graph for 10 epochs. While validation accuracy of feature extraction remains high at 90-92%, fine-tuning achieves an increase in accuracy from 26.77% to 90.22%. Validation accuracy of fine-tuning begins low because the chosen pre-trained layers need to be trained with a small learning rate.

## 15.12 Training and Validation Loss: Feature Extraction vs. Fine-Tuning

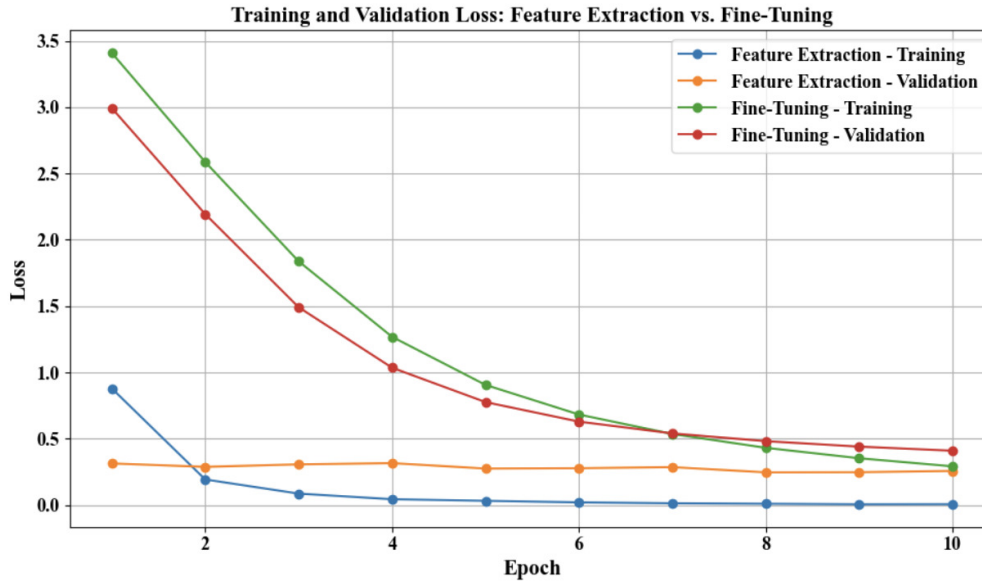


Figure 12: Training and Validation Loss: Feature Extraction vs. Fine-Tuning

### Interpretation:

Losses for training and validation have been compared for the model with and without fine-tuning. The loss obtained from feature extraction is significantly lower with 0.0067 for training and 0.2570 for validation; on the other hand, for fine-tuning, loss reduces to 0.2916 for training and 0.4096 for validation. It can be seen that feature extraction works better in a span of 10 epochs.



### 15.13 5-Fold Cross-Validation Accuracy

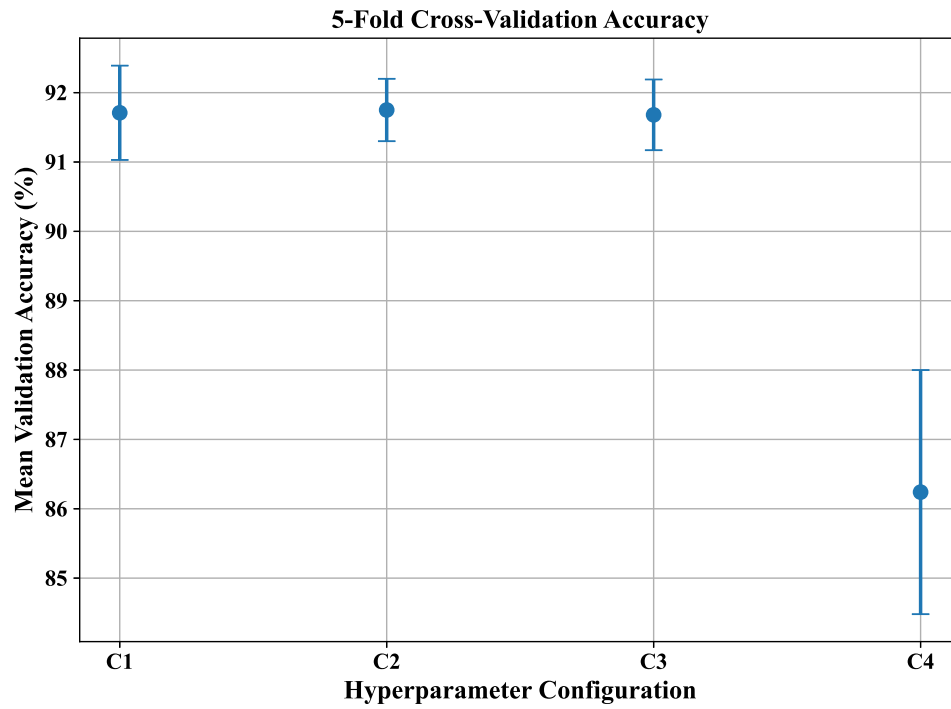


Figure 13: 5-Fold Cross-Validation Accuracy

#### Interpretation:

In the plot, the validation accuracy means and standard deviations of the four hyperparameter settings are compared using five folds. C1, C2, and C3 have a similar level of accuracy of approximately 91.7%. C2 is the best in terms of the accuracy mean value (91.75%) and low variability ( $\pm 0.45\%$ ).

## 15.14 Confusion Matrix

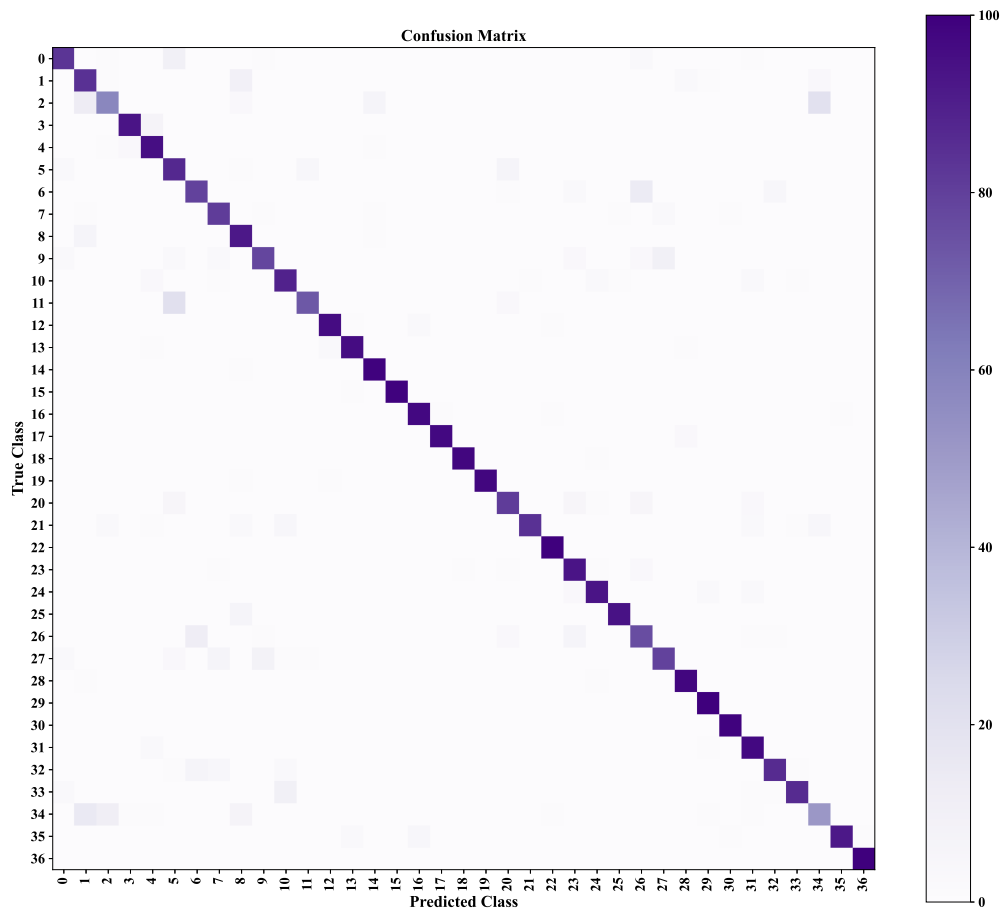


Figure 14: Confusion Matrix

### Interpretation:

From the confusion matrix it is clear that most of the predictions have come along the diagonal path. This implies that the prediction has performed well over all the 37 classes. Misclassification is only seen in case of classes which look alike, i.e. dog breeds like American Pit Bull Terrier and Staffordshire Bull Terrier and cat breeds which also look alike.

## 15.15 Misclassified Images

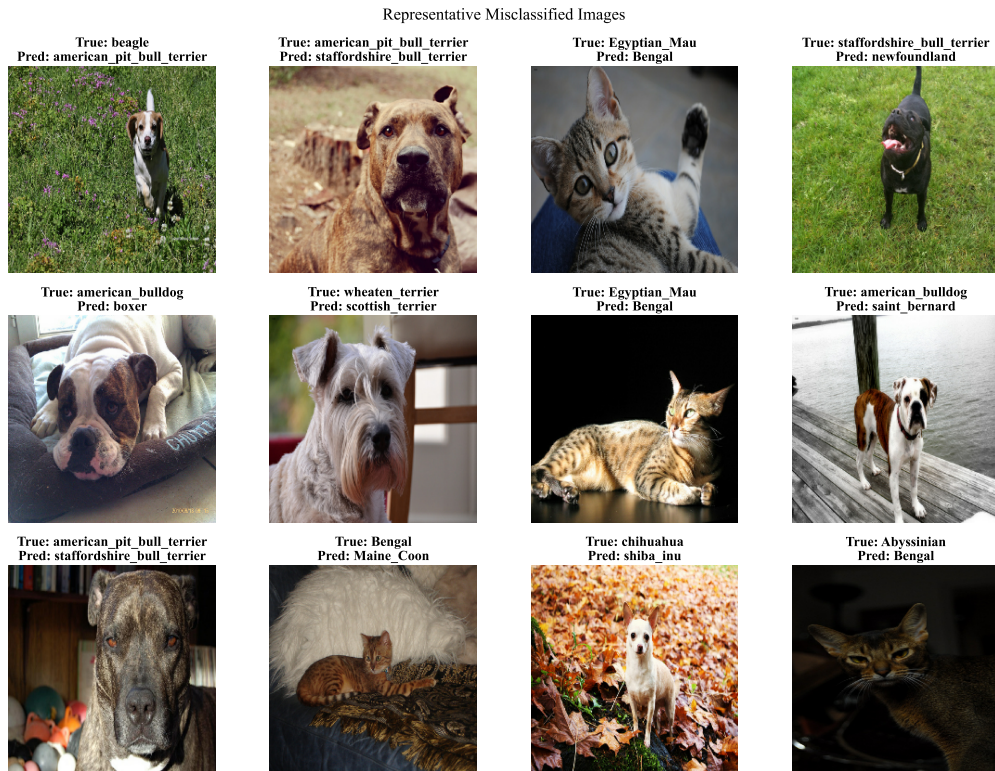


Figure 15: Misclassified Images

### Interpretation:

Plot showcases examples of misclassified images and suggests that misclassification mostly happens among those dog and cat breeds that look alike. For instance, Beagle is classified as American Pit Bull Terrier, and Egyptian Mau is classified as Bengal due to some visual resemblance.

## 16. Discussion Questions

### 1. What is the difference between model parameters and hyperparameters?

Model Parameters refer to the automatically acquired value during model training process, for example, weights and biases. Hyperparameters refer to values chosen before the training process begins, like learning rate, batch size, dropout rate, etc.

### 2. Why is weight initialization important?

Weight Initialization refers to the process of selecting the initial values of the weights of the neural networks. Weight initialization should help the neural network to learn faster and avoid issues like vanishing gradient and symmetric neurons.

### 3. Why can zero initialization be problematic for neural networks?

Since the initial values of all the weights are set to zero, all neurons learn the same feature since they have the same gradient. This makes it impossible for neurons to learn different representations.

### 4. Compare Xavier and He initialization.

Xavier initialization is applicable mostly for sigmoid and tanh activation functions and maintains the right level of activation variance. He initialization technique is applicable mostly for ReLU-based networks and is usually more effective for ReLU activations.

### 5. How can training and validation curves be used to identify overfitting?

In case training accuracy keeps increasing while validation accuracy starts decreasing or remains stagnant, there may be an issue of overfitting. Moreover, in case training loss continues decreasing and validation loss starts increasing, the model is also fitting itself into training data excessively.

### 6. How does Dropout reduce overfitting?

Dropout technique randomly switches off a certain portion of neurons while training. This helps the neural network not to be overly dependent on specific neurons.

### 7. What is the purpose of Batch Normalization?

Normalization within a mini-batch is known as Batch Normalization. It enables stabilization and accelerates the

training process.

8. **Explain the numerical Batch Normalization example.**

For  $x = [2, 4, 6, 8]$ , the batch mean is

$$\mu_B = 5$$

and the batch variance is

$$\sigma_B^2 = 5.$$

Therefore, the normalized values are approximately

$$\hat{x} = [-1.342, -0.447, 0.447, 1.342].$$

Thus, Batch Normalization produces approximately zero-centred and unit-scaled activations.

9. **What are the roles of  $\gamma$  and  $\beta$ ?**

$\gamma$  is a learnable scale parameter, while  $\beta$  is a learnable shift parameter. The output of Batch Normalization is

$$y_i = \gamma \hat{x}_i + \beta.$$

They allow the network to learn an appropriate scale and shift for the normalized activations.

10. **Compare SGD, Momentum, RMSProp and Adam.**

The updates for SGD are made based on the current gradient. Momentum incorporates information from the past updates to speed up learning in a productive manner. RMSProp adjusts the learning rate using the recent squared gradients. Adam incorporates information like momentum for first moment and adaptive learning for the second moment. Adam attained the highest accuracy of around 92.80%.

11. **What happens when the learning rate is too large?**

The weights update becomes very large, and the model might overshoot the minimum. This will lead to instability or non-convergence in training.

12. **What happens when the learning rate is too small?**

The learning rate is extremely low, causing slow convergence to a decent solution, and hence many training iterations.

13. **What is the effect of increasing batch size?**

Larger batches lead to a better estimation of gradients and increase the efficiency of computation. However, too large batch sizes might lead to the reduction of positive randomness in updating parameters and might even worsen the generalization. The best result for the validation set was obtained using the batch size of 64.

14. **Explain stride and padding.**

Stride is the number of pixels by which the convolution filter moves across the input. Padding adds extra pixels, usually zeros, around the input boundaries. The convolution output dimension is

$$O = \left\lfloor \frac{N + 2P - K}{S} \right\rfloor + 1,$$

where  $N$  is the input size,  $K$  is the kernel size,  $P$  is the padding and  $S$  is the stride.

15. **Why is MobileNetV2 computationally efficient?**

MobileNetV2 relies on depthwise separable convolutions, which make the model computationally efficient due to their reduced number of computations and parameters compared to regular convolutions.

16. **What is depthwise separable convolution?**

In depthwise separable convolution, the regular convolution operation is divided into two parts: depthwise convolution and pointwise convolution. The depthwise convolution performs the filtering process for each individual channel, whereas pointwise convolution uses  $1 \times 1$  convolution operations for combining the channels.

17. **What is transfer learning?**

This process involves taking the information that has been learned from the model using a large data set and then applying it to a new task. In this experiment, MobileNetV2 trained using ImageNet is used for the classification of 37 Oxford-IIIT Pet classes.

18. **Differentiate feature extraction and fine-tuning.**

Feature extraction entails freezing the pre-trained MobileNetV2 model and training the new classifier while fine-tuning involves unfreezing some selected higher layers of the pre-trained MobileNetV2 model along with the classifier. From this experiment, feature extraction resulted in better validation performance during the 10 epochs test.

19. **Why is a smaller learning rate generally used during fine-tuning?**

These layers have learned certain useful features. A high learning rate can alter the learned weights, causing damage to the useful pretrained features. Thus, a low learning rate is chosen to allow for incremental changes.

20. **Why is K-Fold Cross-Validation useful for hyperparameter selection?**

K-Fold Cross-Validation evaluates a configuration using multiple train-validation splits. It provides a more reliable estimate of performance and helps determine whether a configuration performs consistently across different subsets of the data.

The selected C2 configuration achieved

$$91.75\% \pm 0.45\%$$

across the five folds.

21. **Why must the test set remain untouched during tuning?**

The testing set must be composed of data that is totally unknown to the algorithm. The use of a testing set for multiple iterations when searching for the optimal hyperparameters will result in its knowledge leaking into the training process.

22. **Why should mean and standard deviation both be reported?**

The mean shows the average performance across the folds, while the standard deviation shows the variation in performance between the folds. For example,

$$91.75\% \pm 0.45\%$$

indicates good average performance with relatively low variation.

23. **Is the highest validation accuracy always sufficient to select a model?**

No, accuracy obtained during validation is not enough. Also, the model will be evaluated based on its mean cross-validation accuracy, standard deviation, training time, tendency to overfit, and performance on independent test set.

In this study, configuration C2 was chosen since it demonstrated 91.75% mean cross-validation accuracy with small standard deviation of 0.45%. This configuration also needed small computational time for training and showed 89.42% accuracy on independent test set.

24. **Why does fine-tuning normally require a smaller learning rate?**

This is because fine-tuning involves the use of a lower learning rate since the model has learned some features from the previous task. This helps preserve the learned representations in the process of adjusting the parameters of the network to suit the new task.

## 17. Additional Exercise

Two new configurations were chosen and tested using the 5 fold Cross-validation method. The outcomes were compared to configuration C2 which was already chosen using mean accuracy, standard deviation, and computational time.

| Configuration | Learning Rate | Dropout | Batch Size | CV Accuracy | SD    |
|---------------|---------------|---------|------------|-------------|-------|
| C2            | 0.001         | 0.25    | 64         | 91.75%      | 0.45% |
| N1            | 0.0001        | 0.50    | 32         | 87.94%      | 1.33% |
| N2            | 0.001         | 0.50    | 64         | 91.47%      | 0.97% |

| Configuration | Fine-Tuning Strategy | Training Time | Test Accuracy |
|---------------|----------------------|---------------|---------------|
| C2            | Frozen Base          | 162.81 s      | 89.42%        |
| N1            | Frozen Base          | 987.49 s      | —             |
| N2            | Partial Unfreezing   | 1012.46 s     | —             |

### Inference:

The selected configuration C2 obtained the highest mean cross validation accuracy of 91.75% and lowest standard deviation of 0.45%. The second best accuracy of 91.47% was obtained by N2, but its training time is relatively higher.

N1 did worse compared to N2 with 87.94% accuracy, with a high standard deviation of 1.33%. Thus, neither of the two configurations is better than configuration C2 since it gives more accuracy, consistency, and reduced computation cost.

C2 final test accuracy is 89.42%, suggesting that C2 does generalize well to an independent test set. Overall, C2 is still the optimal choice, as it achieves the best compromise among all the factors mentioned.

## 18. Expected Outcome

Students should experimentally demonstrate that CNN performance is strongly influenced by initialization, regularization, optimization and hyperparameter choices. They should understand the MobileNetV2 architecture, perform

transfer learning and fine-tuning, and select a reliable final configuration using 5-fold cross-validation.

## 19. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Sergey Ioffe and Christian Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” ICML, 2015.
3. Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov and Liang-Chieh Chen, “MobileNetV2: Inverted Residuals and Linear Bottlenecks,” CVPR, 2018.
4. Omkar M. Parkhi, Andrea Vedaldi, Andrew Zisserman and C. V. Jawahar, “Cats and Dogs,” IEEE Conference on Computer Vision and Pattern Recognition, 2012.
5. TensorFlow Documentation: <https://www.tensorflow.org>
6. Keras Documentation: <https://keras.io>